

ALGORITHM

Spring 2022, Midterm Exam, April 28

5:00 – 8:00 pm

1. (20%) Illustrate the operation of the following sorting algorithms respectively on the array $A = \langle 3, 7, 1, 4, 0, 6, 4, 2, 1, 3, 4 \rangle$, where $A[j] \in \{0, 1, \dots, 7\}$ for $1 \leq j \leq 11$.

Which of them are stable sorting algorithms? Which of them are in-place sorting algorithms?

- (a) MERGE SORT
- (b) BUBBLE SORT
- (c) INSERTION SORT
- (d) QUICKSORT

2. (20%) Let $f(n)$ and $g(n)$ be real-valued functions. The asymptotic notations, O , Ω , and Θ , are defined as follows.

- $f(n)$ is asymptotically upper bounded by $g(n)$, denoted by $f(n) = O(g(n))$,
iff there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$.
- $f(n)$ is asymptotically lower bounded by $g(n)$, denoted by $f(n) = \Omega(g(n))$,
iff there exists positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n > n_0$.
- $f(n)$ is asymptotically equivalent to $g(n)$, denoted by $f(n) = \Theta(g(n))$,
iff there exists positive constants c_1 , c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ whenever $n > n_0$.

Please answer the following questions according to the definitions given above.

- (a) Finding upper bound for the function $f(n) = n^3 - 6n + 2$ by using the O notation.
Justify your answer by demonstrating the constants.
- (b) Finding lower bound for the function $f(n) = n^3 - 6n + 2$ by using the Ω notation.
Justify your answer by demonstrating the constants.
- (c) Finding tight bound for the function $f(n) = n^3 - 6n + 2$ by using the Θ notation.
Justify your answer by demonstrating the constants.
- (d) Show that $n^3 - 6n + 2 \neq \Omega(n^4)$.

3. (15%) Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 3T(\lfloor n/3 \rfloor) + n$, where $T(1) = 3$ and $T(2) = 6$. Use the substitution method to verify your answer.

4. (10%) Given a max heap in an array $A[1 \dots n]$ with $A[1]$ the maximum key, give pseudo code to implement the following routine, while maintaining the max heap property.
DECREASE-KEY(i, δ): Decrease the value of the key currently at $A[i]$ by $\delta \geq 0$.

5. (15%) Use the master method to give tight asymptotic bounds for the following recurrences.

(a) $T(n) = 2T(n/3) + n \log n$.

(b) $T(n) = 8T(n/2) + n^3$.

(c) $T(n) = 16T(n/4) + n^3$.

6. (15%) A lazy programmer thinks that implementing the procedure MERGE (of the merge sort) is too complicated. To simplify MERGE-SORT he eliminates the merge all together, and only recursively "sorts" the two halves of the array. Here is his pseudo code.

LAZY-SORT(A, p, r)

```
1  if  $p < r$ 
2       $q \leftarrow \lfloor (p+r)/2 \rfloor$ 
3      LAZY-SORT( $A, p, q$ )
4      LAZY-SORT( $A, q+1, r$ )
```

Initial call: LAZY-SORT($A, 1, n$)

- (a) What does LAZY-SORT do?.
- (b) What is the running time of LAZY-SORT?
7. (15%) Given "sorted array" of n distinct integers, arranged in increasing order $A[1 \dots n]$, you want to find out whether there is an index i for which $A[i] = i$. Give an algorithm that runs in time $O(\log n)$ for this problem.